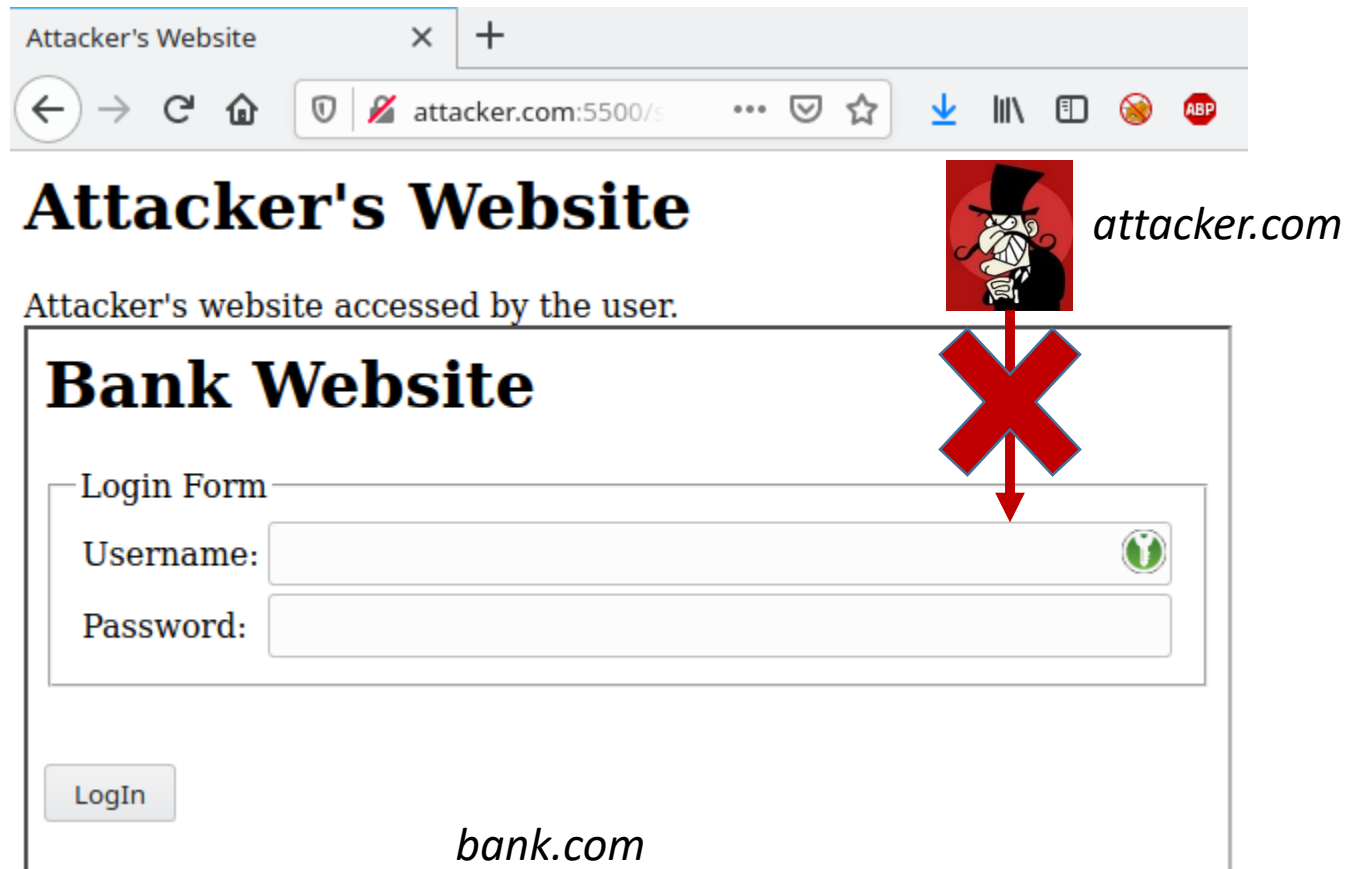


Injection Attacks

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Same Origin Policy (SOP)



Bypassing SOP: Injecting script directly into the website

Bank Website

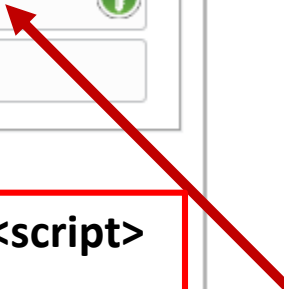
Login Form

Username:

Password:

bank.com

`<script>
....
</script>`



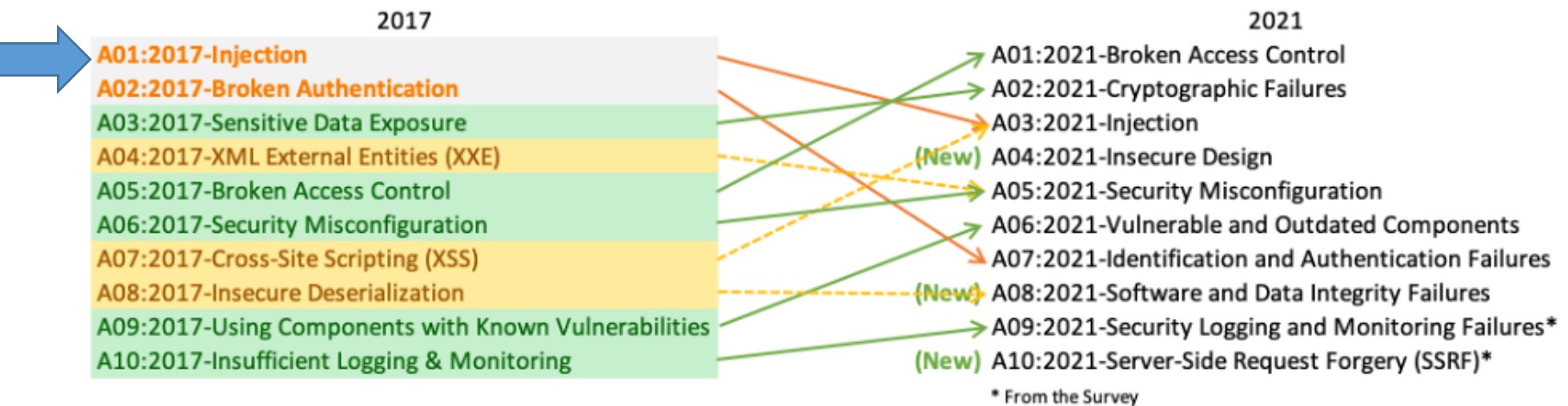
Attacks until now

- Reflected XSS
- Stored XSS
- DOM-based XSS
- mXSS
- Scriptless attacks
- DOM clobbering
- UI redressing



**Today: let's talk about
(other) injections**

Motivation – Injections in OWASP Top 10



What injection attacks do you know?

Example: Command Injection

- Vulnerable PHP-Code (delete.php)

```
<?php
    print("Please specify the file to delete<br>");
    $file=$_GET['filename'];
    system("rm $file");
?>
```

„;“ closes the data-context correctly
The next strings are bash commands

- Request

http://127.0.0.1/delete.php?filename=bob.txt;id

- Response

Please specify the file to delete
uid=33(www-data) gid=33(www-data) groups=33(www-data)

There are many injection attacks. Today,
we talk about **SQL** injections.

SQL Basics

SQL (Structured Query Language)

- Standardized in 1986
- The most commonly used database programming language:
 - Oracle PL/SQL
 - MySQL
 - Microsoft SQL
- Used by Google, Facebook, ...

Databases – Ranking

420 systems in ranking, May 2024

Rank			DBMS	Database Model	Score		
May 2024	Apr 2024	May 2023			May 2024	Apr 2024	May 2023
1.	1.	1.	Oracle 	Relational, Multi-model 	1236.29	+2.02	+3.66
2.	2.	2.	MySQL 	Relational, Multi-model 	1083.74	-3.99	-88.72
3.	3.	3.	Microsoft SQL Server 	Relational, Multi-model 	824.29	-5.50	-95.80
4.	4.	4.	PostgreSQL 	Relational, Multi-model 	645.54	+0.49	+27.64
5.	5.	5.	MongoDB 	Document, Multi-model 	421.65	-2.31	-14.96
6.	6.	6.	Redis 	Key-value, Multi-model 	157.80	+1.36	-10.33
7.	7.	 8.	Elasticsearch	Search engine, Multi-model 	135.35	+0.57	-6.28
8.	8.	 7.	IBM Db2	Relational, Multi-model 	128.46	+0.97	-14.56
9.	9.	 11.	Snowflake 	Relational	121.33	-1.87	+9.61
10.	10.	 9.	SQLite 	Relational	114.32	-1.69	-19.54

- The attacks described in this lecture applicable to all SQL/relational databases (and even beyond)

SQL databases store data in tables

- Example: bank database **CUSTOMERS**

ID	USER	BALANCE	PASSWORD
1	alice	10000	12345
2	bob	20000	password
3	charlie	30000	xyz

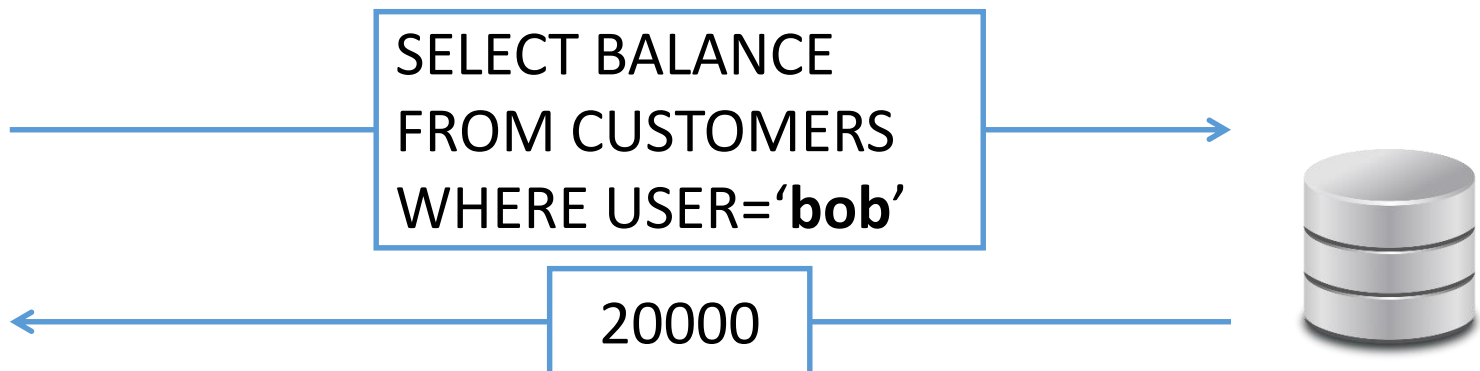
This is just an example! Do not use plain passwords in databases!



Querying data from CUSTOMERS

ID	USER	BALANCE	PASSWORD
1	alice	10000	12345
2	bob	20000	password
3	charlie	30000	xyz

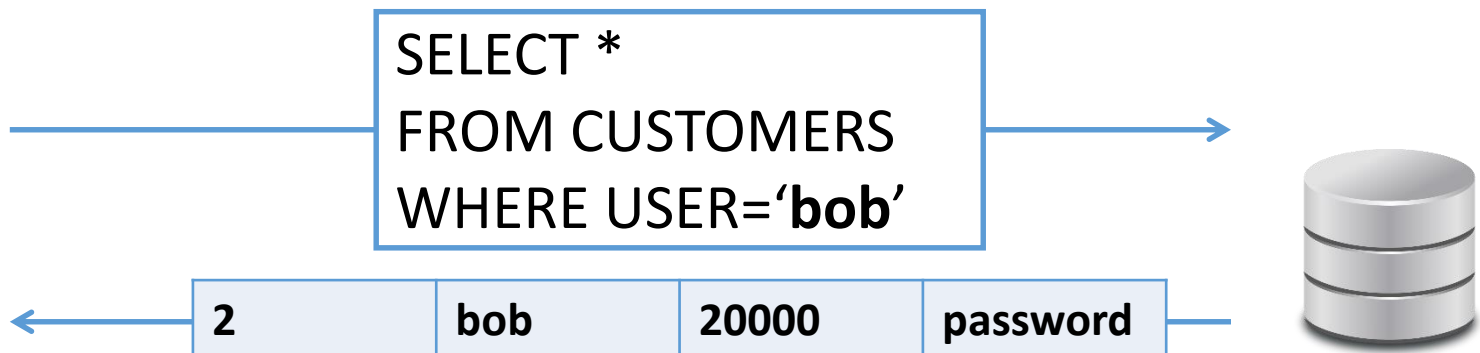
- Example: Retrieving data for bob:



Querying data from CUSTOMERS

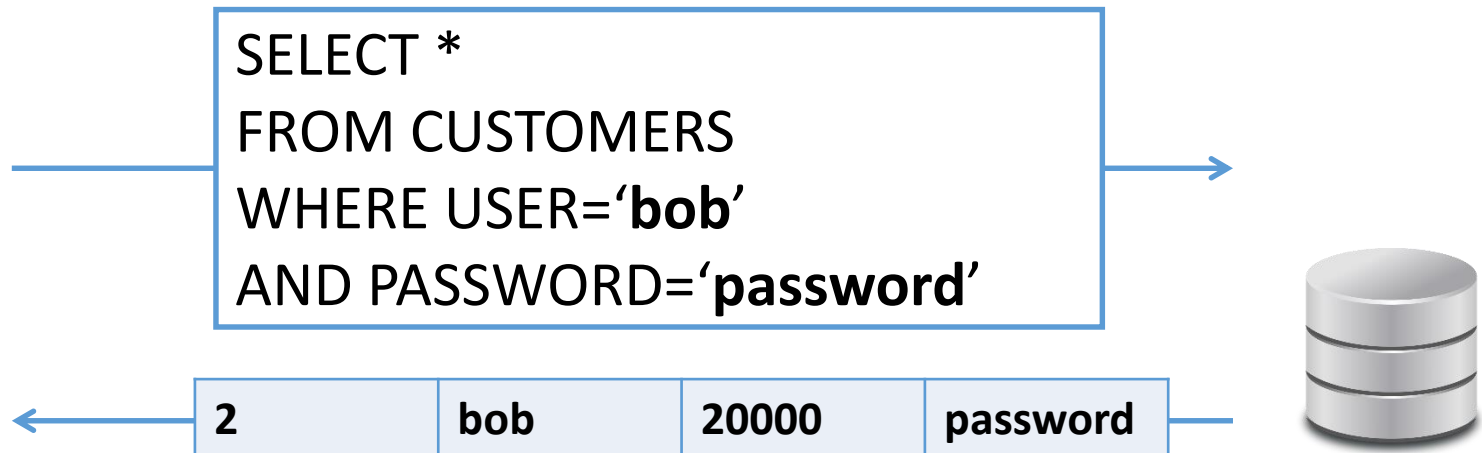
ID	USER	BALANCE	PASSWORD
1	alice	10000	12345
2	bob	20000	password
3	charlie	30000	xyz

- Example: Retrieving data for bob:



SQL operators (AND, OR, NOT)

ID	USER	BALANCE	PASSWORD
1	alice	10000	12345
2	bob	20000	password
3	charlie	30000	xyz



SQL commands

- Data manipulation:
 - SELECT, INSERT, UPDATE, DELETE
- Data definition:
 - CREATE, ALTER, DROP
- Data control:
 - GRANT, REVOKE

- Example:

```
CREATE TABLE employees(  
    userid varchar(6) not null primary key,  
    first_name varchar(20),  
    last_name varchar(20),  
    department varchar(20),  
    salary varchar(10)  
);
```

DEMO

Questions



Injection Attacks

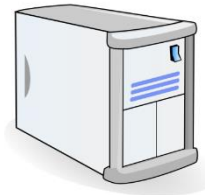
Injection Attacks

Injection attacks allow
a ~~programmer~~ user specified query
to be executed in the database

Motivation: What could go wrong?



http://search.com?search=mustermann



search.com

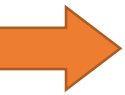
```
$search = $_REQUEST['search'];  
$query = "SELECT * FROM Persons WHERE LastName='$search'";
```

← → ↻ 🏠 🔍 search.com/?search=mustermann 📖 🚫 ⏏

Submit

Array ([PersonID] => 1 [0] => 1 [LastName] => mustermann [1] => mustermann [FirstName] => erika [2] => erika [Address] => somestreet [3] => somestreet [City] => ny [4] => ny)

Injection Attacks: Agenda

- 
1. Attacker Model
 2. SQL Injection Attacks
 3. Other (more) Injection Types
 4. Countermeasures and BCPs

Injection Attacks: Attacker Model

- Prerequisites
 - The web application has a data repository
 - The attacker can communicate with the web application
 - The attacker influences (partially) the database query

Injection Attacks: Attacker Model

- Attackers Goal
 - Denial-of-Service
 - Unauthorized read/write access to data
 - Determining database schema
 - Extracting data
 - Adding or modifying data
 - Bypass authentication

Injection Attacks

- There are different attack types
 - Command Injection
 - SQL Injection
 - LDAP Injection
 - XPath Injection
 - Log Injection
 - ...
- XSS is also a form of an injection attack
- We concentrate on **SQL injection** attacks

Injection Attacks: Agenda

1. Attacker Model
-  2. SQL Injection Attacks
3. Other (more) Injection Types
4. Countermeasures and BCPs

SQL injection

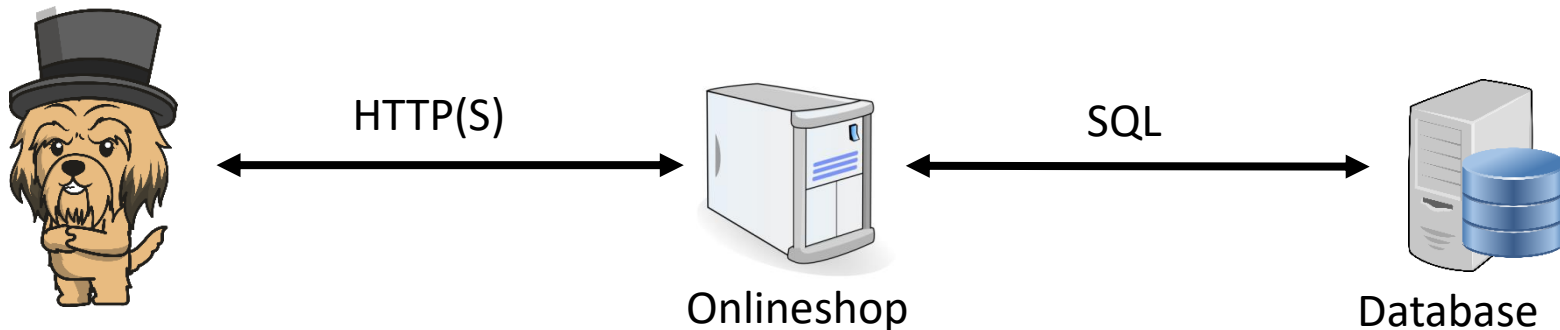
- General problem:
 - Mixing of SQL commands with user input data
- Used to
 - read and store data
 - perform Denial of Service
 - influence the application logic
 - inject malicious content into application

SQL Injection

- In 1998, Phrack Magazine mentioned the problem for the first time:

„Don't assume user's input is ok for SQL queries“,

<http://www.phrack.org/archives/issues/54/8.txt>



Injection Attacks: Goals

- Attackers Goal



- Denial-of-Service
- Unauthorized read/write access to data
 - Information leakage
 - Extracting data
 - Adding or modifying data
 - Bypass authentication

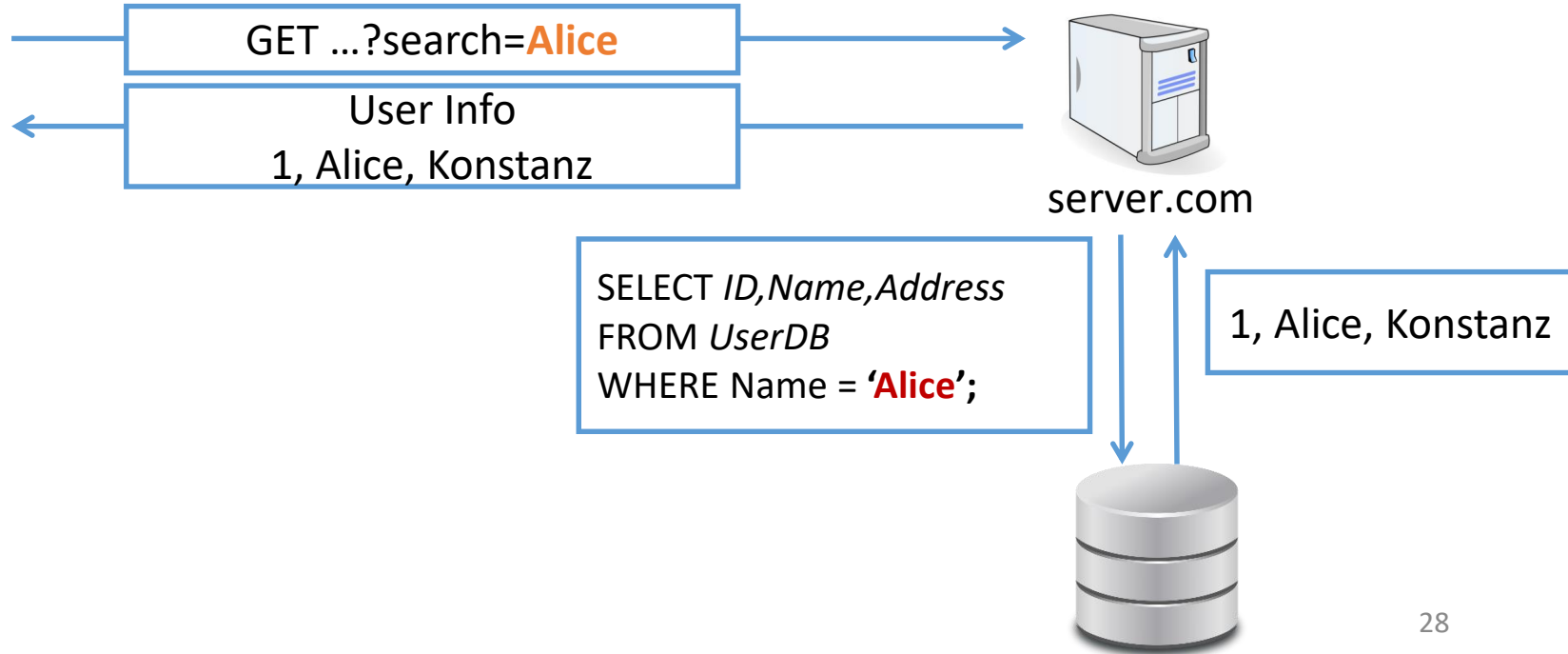
Scenario example

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



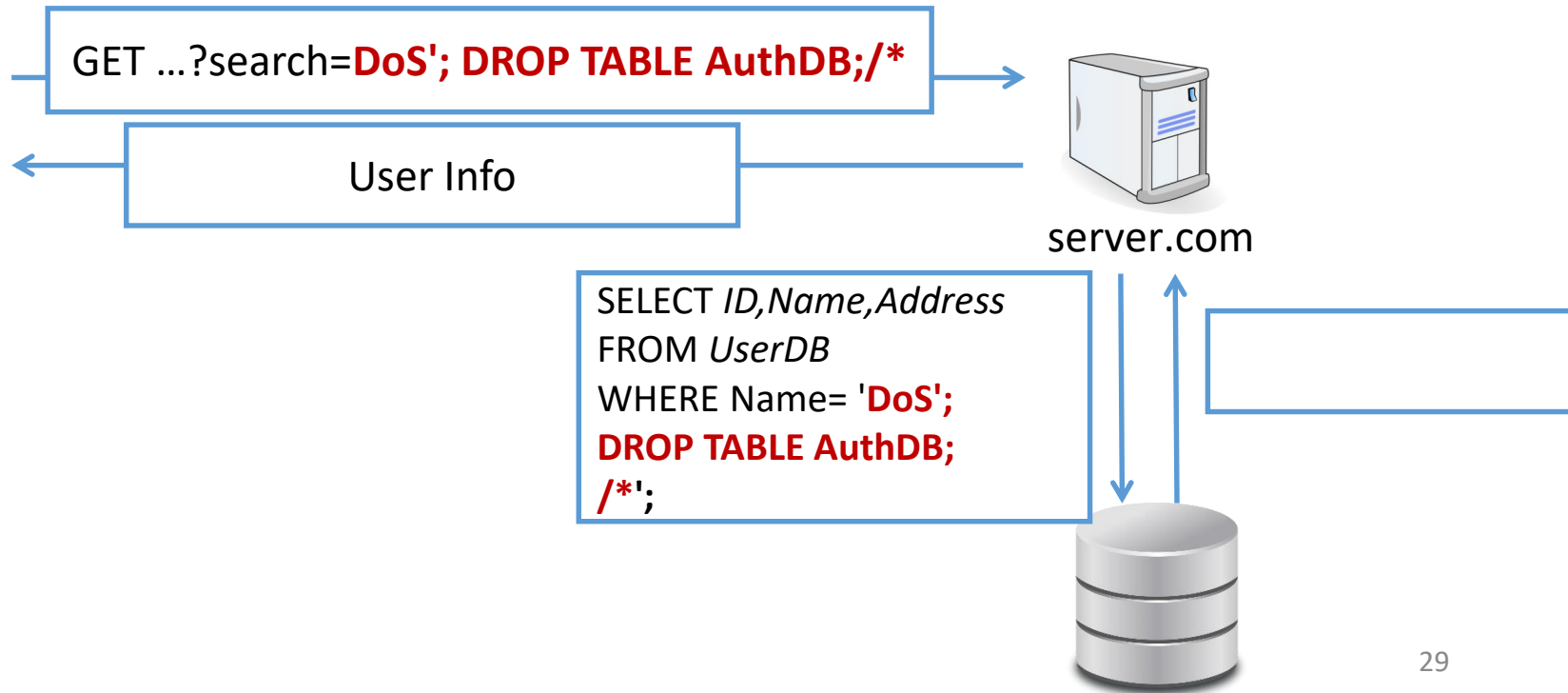
Injection Attacks: Denial-of-Service

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



Injection Attacks: Denial-of-Service

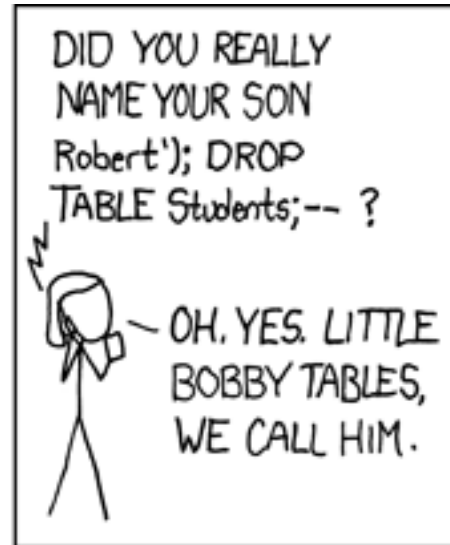
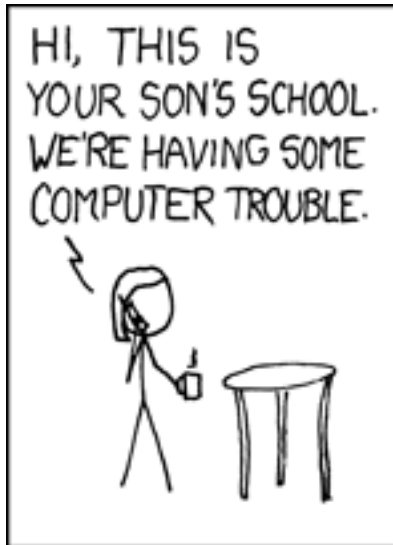
- Denial-of-Service Attacks
 - Delete all databases: DROP TABLE *name*
 - Copy existing large tables

```
CREATE TABLE LargeTableCopy AS SELECT * FROM LargeTable;
```

- Do expensive operations by
 - Returning big data blobs (network resources)
 - Computing/Sorting/Merging big data blobs (CPU/memory resources)

```
SELECT BENCHMARK(10000000, sha1('test'));
```

Questions



Source: <http://xkcd.com/327/>

Injection Attacks: Goals

- Attackers Goal
 - Denial-of-Service
 - Unauthorized read/write access to data
 - Information leakage
 - Extracting data
 - Adding or modifying data
 - Bypass authentication



Injection Attacks: Information Leakage

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

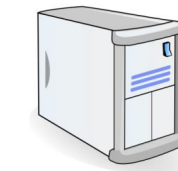
ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



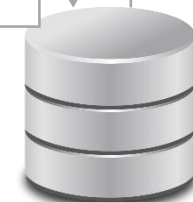
GET ...?search=DoS'; DROP TABLE AuthDB; /*

User Info



server.com

```
SELECT ID,Name,Address
FROM UserDB
WHERE Name= 'DoS';
DROP TABLE AuthDB;
/*;
```



Attack successful or not?

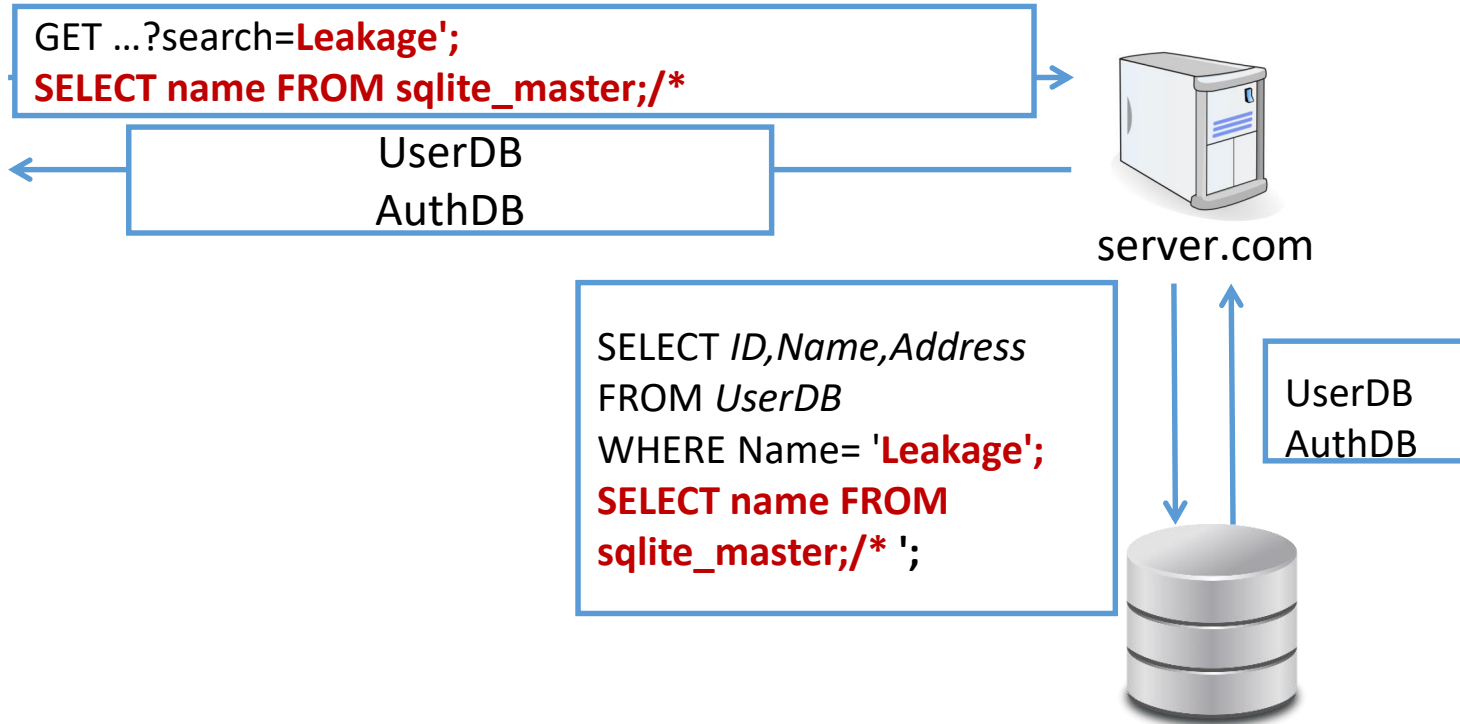
Injection Attacks: Information Leakage

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



Injection Attacks: Information Leakage

- SQLite: *sqlite_master*
 - Example: *SELECT name FROM **sqlite_master**;*
 - Example: *SELECT sqlite_version();*
- MySQL/MariaDB: *information_schema.tables*
 - Example: *SELECT table_name FROM **information_schema.tables**;*
 - Example: *SHOW VARIABLES LIKE "%**version**%";*
 - *version_ssl_library/version_compile_os/innodb_version/protocol_version*
- PostgreSQL: *pg_catalog*
 - Example: *SELECT * FROM pg_catalog.pg_tables*
 - Example: *SELECT version();*

Injection Attacks: Goals

- Attackers Goal
 - Denial-of-Service
 - Unauthorized read/write access to data
 - Information leakage
 - Extracting data
 - Adding or modifying data
 - Bypass authentication



Injection Attacks: Extracting data

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

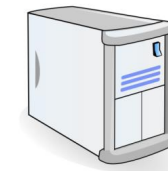
ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



GET ...?search=**Attack'**;
SELECT * FROM UserDB; /*

[UserDB content]



server.com

```
SELECT ID,Name,Address
FROM UserDB
WHERE Name = 'Attack'';
SELECT * FROM UserDB;
/*;
```

ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



Injection Attacks: Extracting data

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

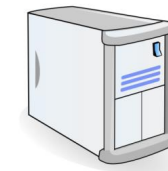
ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

UserDB



GET ...?search=**Attack'**;
SELECT * FROM AuthDB; /*

[AuthDB content]



server.com

```
SELECT ID,Name,Address
FROM UserDB
WHERE Name = 'Attack'';
SELECT * FROM AuthDB;
/*;
```

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB



Questions



Injection Attacks: Goals

- Attackers Goal
 - Denial-of-Service
 - Unauthorized read/write access to data
 - Information leakage
 - Extracting data
 - Adding or modifying data
 - Bypass authentication



Injection Attacks: Data Manipulation

ID	username	password
1	alice	12345
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

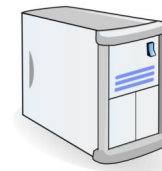
ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Administrator

UserDB



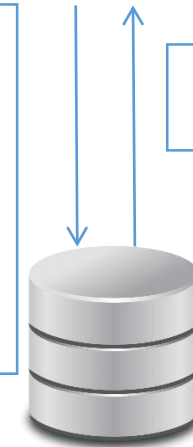
```
GET ...?search=Attack';  
UPDATE UserDB SET Role = "Administrator" WHERE ID=4; /*
```

User Info



server.com

```
SELECT ID,Name,Address  
FROM UserDB  
WHERE Name = 'Attack';  
UPDATE UserDB SET Role =  
"Administrator" WHERE  
ID=4;  
/* ';
```



Injection Attacks: Data Manipulation

ID	username	password
1	alice	1
2	bob	password
3	charlie	xyz
4	attacker	evil

AuthDB

ID	Name	Address	Role
1	Alice	Konstanz	Professor
2	Bob	Paderborn	PhD
3	Charlie	Wuppertal	Administrator
4	Attacker	Bochum	Student

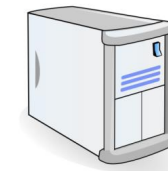
UserDB



GET ...?search= **Attack'**;

UPDATE AuthDB SET password = "1" WHERE ID=1;/*

User Info



server.com

```
SELECT ID,Name,Address
FROM AuthDB
WHERE Name = 'Attack';
UPDATE AuthDB SET
password = "1" WHERE
ID=1;
/*';
```



Injection Attacks: Goals

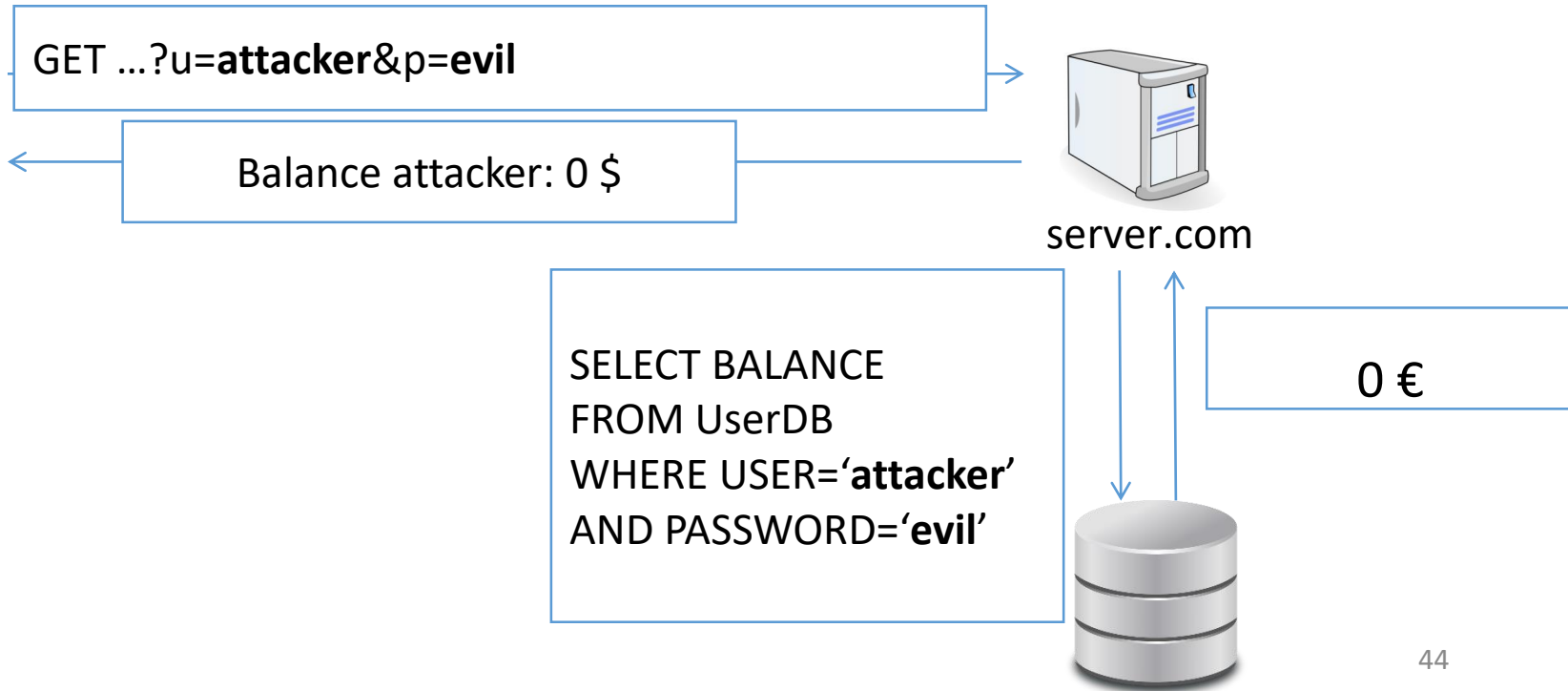
- Attackers Goal
 - Denial-of-Service
 - Unauthorized read/write access to data
 - Information leakage
 - Extracting data
 - Adding or modifying data
 - Bypass authentication



Injection Attacks: Authentication Bypass

ID	Username	Password	Address	Role	Balance
1	alice	12345	Konstanz	Professor	10.000 €
2	bob	password	Paderborn	PhD	20.000 €
3	charlie	xyz	Wuppertal	Administrator	30.000 €
4	attacker	evil	Bochum	Student	0 €

UserDB



Injection Attacks: Authentication Bypass

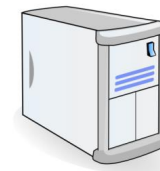
ID	Username	Password	Address	Role	Balance
1	alice	12345	Konstanz	Professor	10.000 €
2	bob	password	Paderborn	PhD	20.000 €
3	charlie	xyz	Wuppertal	Administrator	30.000 €
4	attacker	evil	Bochum	Student	0 €

UserDB



GET ...?u=attacker&p=evil

Balance attacker: 0 \$



server.com

Imagine this data processing:

q = "SELECT BALANCE FROM CUSTOMERS
WHERE USER=" + u + " AND PASSWORD = " + p + ";"



Injection Attacks: Authentication Bypass

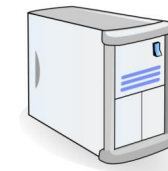
ID	Username	Password	Address	Role	Balance
1	alice	12345	Konstanz	Professor	10.000 €
2	bob	password	Paderborn	PhD	20.000 €
3	charlie	xyz	Wuppertal	Administrator	30.000 €
4	attacker	evil	Bochum	Student	0 €

UserDB



GET ...?u=**bob**&p=**x ' or '1'='1**

Balance bob: 20.000 €



server.com

```
SELECT BALANCE
FROM UserDB
WHERE USER='bob' AND
PASSWORD='x ' or '1'='1'
```

20.000 €



DEMO

Last part of the SQL injection

```
"SELECT * FROM user_data WHERE first_name = 'John' AND last_name = '' + lastName + ''";
```



<https://pingo.coactum.de/633763>

- A) 1=1
- B) 1=2
- C) 1'='2
- D) '1'='1
- E) '1'='2
- F) Last_name='Smith'

SQL Injection – HTTP POST

- Form fields could be susceptible (e.g., radio buttons or select lists)

```
<select name=„choice">  
  <option value="1">small</option>  
  <option value="2">medium</option>  
  <option value="' ;/*">SQL Injection</option>  
</select>
```

SQL Injection – HTTP Cookies

- Cookies could be directly used within SQL-Statements

```
GET /alice HTTP/1.0
Host: www.wonderland.org
Content-type: text/html
Cookie: MyCookie=AYQEVn...DKrdst' ;/*
```

- Basically, all header fields could be used as an injection point

Legal note



**Do NOT try these attacks
on real websites!**

Questions (?)

10 minute break – try to log in as an admin ...

<https://demo.testfire.net/login.jsp>

Injection Attacks: Agenda

1. Attacker Model
2. SQL Injection Attacks
-  3. Other (more) Injection Types
4. Countermeasures and BCPs

Real-world problem?



Common Vulnerabilities and Exposures

[CVE List](#)

[CNAs](#)
[About](#)

[WGs](#)
[News & Blog](#)

[Board](#)

NVD

Go to for:

[CVSS Scores](#)

[CPE Info](#)

[Advanced Search](#)

[Search CVE List](#)

[Download CVE](#)

[Data Feeds](#)

[Request CVE IDs](#)

[Update a
CVE Entry](#)

TOTAL CVE Entries: 134859

[HOME](#) > [CVE](#) > [SEARCH RESULTS](#)

Search Results

There are **8282** CVE entries that match your search.

Name	Description
CVE-2020-9521	An SQL injection vulnerability was discovered in Micro Focus Service Manager Automation (SMA), affecting versions 2019.08, 2019.05, 2019.02, 2018.08, 2018.05, 2018.02. The vulnerability could allow for the improper neutralization of special elements in SQL commands and may lead to the product being vulnerable to SQL injection.
CVE-2020-9465	An issue was discovered in EyesOfNetwork eonweb 5.1 through 5.3 before 5.3-3. The eonweb web interface is prone to a SQL injection, allowing an unauthenticated attacker to perform various tasks such as authentication bypass via the user_id field in a cookie.
CVE-2020-9402	Django 1.11 before 1.11.29, 2.2 before 2.2.11, and 3.0 before 3.0.4 allows SQL Injection if untrusted data is used as a tolerance parameter in GIS functions and aggregates on Oracle. By passing a suitably crafted tolerance to GIS functions and aggregates on Oracle, it was possible to break escaping and inject malicious SQL.
CVE-2020-9398	ISPConfig before 3.1.15p3, when the undocumented reverse_proxy_panel_allowed=sites option is manually enabled, allows SQL Injection.
CVE-2020-9340	fauzantrif eLlection 2.0 has SQL Injection via the admin/ajax/op_kandidat.php id parameter.
CVE-2020-9318	Red Gate SQL Monitor 9.0.13 through 9.2.14 allows an administrative user to perform a SQL injection attack by configuring the SNMP alert settings in the UI. This is fixed in

Real-world exploits

- <http://www.glorf.it/blog/2011/06/15/sql-talk/die-hacks-gegen-sony-sql-injection-uberwiegt>
- Anonymous attacked Sony Online Entertainment in 2011
- Responsible, for example, for Sony Playstation Network
- DoS, XSS, ...and many SQL injections

Real-world exploits

- 2007, Albert Gonzalez uploaded his malware into ATM system
- “data from 130 million transactions had been exposed”
- <https://www.nytimes.com/2010/11/14/magazine/14Hacker-t.html?pagewanted=all>

Real-world exploits

SQL injection vulnerability in e-learning platform Moodle could enable database takeover

Jessica Haworth 08 March 2022 at 14:36 UTC

Updated: 10 March 2022 at 10:32 UTC

Open Source Software

Vulnerabilities

SQL Injection



Security flaw could risk data leak

```
private static function unserializestring($serializedstring)
{
    $variables = array();
    $a = preg_split("/(\\". "\\", $serializedstring, -1, PREG_S
        | PREG_SPLIT_DELIM_CAPTURE);
```



```
    $variables[$a[$i]] = unserialize($a[$i + 1]);
}
return $variables;
```

A security vulnerability in e-learning platform Moodle could allow an attacker to take over a database and potentially obtain sensitive information, researchers have warned.

Moodle is an open source educational resource that enables institutions to create online learning materials for students.

Researchers have found that the website is vulnerable to a second order [SQL injection](#) flaw, which could enable an attacker to potentially take control of a [database](#) server.

<https://portswigger.net/daily-swig/sql-injection-vulnerability-in-e-learning-platform-moodle-could-enable-database-takeover>

Other types of injections

- Blind SQL injection:
 - If no direct backchannel is available
- Attacking NoSQL
- Other channels like OCR scanners...



Find automatically?

- Sqlmap
- SqlNinja
- Havij
- Mole
- SQLSus
- BSQLHacker
- <https://pentest-tools.com/website-vulnerability-scanning/sql-injection-scanner-online>

Questions



Injection Attacks: Agenda

1. Attacker Model
2. SQL Injection Attacks
3. Other (more) Injection Types
-  4. Countermeasures and BCPs

General problem

- Mixing user input with SQL query strings
- Separation needed
- [https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/SQL Injection Prevention Cheat Sheet.md](https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/SQL%20Injection%20Prevention%20Cheat%20Sheet.md)

Injection Attacks: Step-by-Step

- Searching for a vulnerable point
 - https://owasp.org/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/07-Input_Validation_Testing/05-Testing_for_SQL_Injection.html
- Information gathering
 - Fingerprinting the Database
 - Meta-information: OS, used libraries, ...
 - Table list
- Retrieving data
 - Table dumps, username/passwords, ...
- Privilege Escalation and persistent access
 - OS take over
 - Data change

Errors as Information Source

- Every single piece of information is useful for the attacker

An Error Has Occurred

Summary:

Syntax error in string in query expression '1=1 and t.trans_date >= ' and a.userid = 100116014 ORDER BY 1 DESC'.

Error Message:

System.Data.OleDb.OleDbException: Syntax error in string in query expression '1=1 and t.trans_date >= ' and a.userid = 1001
System.Data.OleDb.OleDbCommand.ExecuteNonQueryForSingleResult(tagDBPARAMS dbParams, Object& executeResult) at
System.Data.OleDb.OleDbCommand.ExecuteNonQuery(CommandBehavior behavior, Object& executeResult) at System.Data.Ole
System.Data.OleDb.OleDbCommand.ExecuteReader(CommandBehavior behavior) at System.Data.OleDb.OleDbCommand.System
datatables, Int32 startRecord, Int32 maxRecords, String srcTable, IDbCommand command, CommandBehavior behavior) at Sys

- Disable debugging in production systems

Countermeasures

- Countermeasures for SQL Injection attacks
 - Prepared Statements
 - Validation
 - Escaping
 - Limit DB-rights and restricted SQL-Queries

Prepared Statements

- The most secure variant to prevent SQL Injection
- Separation between query and data context used within an SQL statement
 - Example: `SELECT * FROM customer WHERE name = ?;`
- The data context is marked with „?“
- Quotes are not needed any more
- It is **not** possible to escape the data context by using quotes or semi-colon

Prepared Statements – Insecure

- Insecure Way – the string concatenation

```
public List<Customer> getCustomer(String custName) {  
    String query = "SELECT * FROM customer "  
    + "WHERE name = '" + custName + "'";  
  
    Connection con;  
    // ...  
    PreparedStatement pstmt = con.prepareStatement(query);  
    ResultSet rs = pstmt.executeQuery();  
    // ...  
}
```

- Use always „?“ and *setter* methods

Prepared Statements – Example

- Customer data used in prepared statement *setter* methods

```
public List<Customer> getCustomer(String custName) {  
    String query = "SELECT * FROM customer WHERE name = ?";  
  
    Connection con;  
    // ...  
    PreparedStatement pstmt = con.prepareStatement(query);  
    pstmt.setString(1, custName);  
    ResultSet rs = pstmt.executeQuery();  
    // ...  
}
```

Validation

- Always validate user input (denylist vs allowlist)
 - Use a list with allowed characters, commands
 - Defining a complete denylists is almost impossible
 - Enforce server-side validation
- Validation does not protect 100%
 - Validation reduces the attack surface
 - Sometimes special characters must be allowed
 - Combine all countermeasures

Denylists vs. Allowlists

- Denylists
 - Explicitly disallow characters or strings → hard to be complete
 - Different databases define different characters and commands
 - Obfuscation possible: `' ; DROP /* lala */ TABLE customer; --`
 - Denylists must be updated constantly
- Allowlists → The way to go!
 - Allow that what you need
 - Less susceptible against new attacks

Validation Example

- JSR 349 Bean Validation¹ und Hibernate Validator²

```
public class Customer {  
    @NotNull  
    @Pattern(regexp="^[a-z0-9]+")  
    private String username;  
  
    @Email  
    private String email;  
  
    // ...  
}
```

¹ <http://beanvalidation.org/1.1/spec/>

² <http://www.hibernate.org/subprojects/validator.html>

Escaping

- Encode dangerous characters
 - Example: `0 'Andy` -> `0 \'Andy`
- OWASP Enterprise Security API (ESAPI) with the Encoder Interface¹
 - Note: Depends on the chosen database
- Escaping is needed, if no prepared statements can be used

¹http://owasp-esapi-java.googlecode.com/svn/trunk_doc/latest/org/owasp/esapi/Encoder.html

Limit DB-rights and restricted SQL queries

- For DB users is valid: *Allow as less as possible as much as needed*
 - Use fine-grained databases and tables
 - Allow write access only for a small amount of tables
 - Different users with different access rights
- LIMIT und other SQL-commands to prevent massive data extraction
 - Only limit the impact of already successful attack

Be careful when pentesting

- Many of you (will) do bug bounty programs and perform security evaluations
- Be aware of rules you need to follow:
 - Do not perform DoS
 - Do not access user data, servers
 - Do not run scanners if you do not know how they work
 - Provide proof-of-concepts with harmless content (e.g., `alert(1)`);
- It is not always that easy:
 - Sometimes applications can crash ...
 - Or user data can be revealed



Conclusions

- We learned about XSS, CSRF, Clickjacking, SQL injections, ...
- They exploit complexity of web applications
- Do **NOT** try these attacks on real web applications!
- Be careful when developing your application
 - There is much more
 - Use OWASP cheat sheets
- Next lecture: XML security

Questions



Sources

- [https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Clickjacking Defense Cheat Sheet.md](https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Clickjacking%20Defense%20CheatSheet.md)
- <https://crypto.stanford.edu/~dabo/pubs/papers/framebust.pdf>
- [https://www.owasp.org/index.php/Category:OWASP Top Ten Project](https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project)
- <https://github.com/WebGoat/WebGoat>
- <https://www.youtube.com/watch?v=ciNHn38EyRc>
- <https://www.youtube.com/watch?v=Q-RZXz6SP1o>

Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project



- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov